# UNIX
# 2 MARKS QUESTIONS

---

### 1. How many types of shell in UNIX?

UNIX supports several types of command interpreters known as **shells**, each with its own features and syntax. The main types are:

1. **Bourne Shell (sh)** – the original UNIX shell.
2. **C Shell (csh)** – uses C-like syntax and supports history and aliasing.
3. **Korn Shell (ksh)** – combines features of Bourne and C shells.
4. **Bourne Again Shell (bash)** – most widely used; supports scripting, command editing, and job control.
5. **Z Shell (zsh)** – advanced shell with customization and plugin support.
   Each shell provides an interface between the user and the operating system.

---

### 2. What is a foreground process?

A **foreground process** is one that runs directly under user control in the terminal window. It occupies the terminal until it completes, meaning the user cannot run other commands from that terminal until the process finishes. The process takes input from the keyboard and displays its output or errors on the screen. For example, when you run cat file.txt, it executes in the foreground until you exit.

---

### 3. What do you mean by booting process in UNIX?

The **booting process** in UNIX refers to the sequence of operations performed by the system when it starts. It begins when the computer is powered on, and the BIOS/firmware loads the **bootloader** (like GRUB). The bootloader then loads the **UNIX kernel** into memory. The kernel initializes system hardware, mounts the root file system, and starts essential background processes (daemons). Finally, it provides the **login prompt**, allowing users to access the system.

---

### 4. Define shell variable.

A **shell variable** is a variable defined within the shell environment that stores temporary data or values used by commands and scripts. These variables can store user input, filenames, paths, or configuration details. There are two types of shell variables:

- **Local variables** – available only in the current shell.
- **Environment variables** – available to all child processes.
  Example:

```
name="Ayush"

echo $name
```

---

### 5. What do you understand by GUI scripting?

**GUI scripting** involves automating actions performed in graphical user interfaces (GUIs). Instead of typing commands in the terminal, it simulates human interactions such as mouse clicks, keystrokes, or menu selections on GUI applications. It is often used in **software testing**, **system automation**, and **repetitive tasks**. Tools like xdotool, Sikuli, or Python libraries (like PyAutoGUI) can be used for GUI scripting in UNIX systems.

---

### 6. In what ways does a shell script get input values?

A shell script can receive input in several ways:

1. **Command-line arguments** – Values passed when executing the script, accessed using $1, $2, etc.
2. **User input during execution** – Using the read command to accept data interactively.
3. **Environment variables** – Predefined or user-defined variables accessible within the script. Example:

```
read name

echo "Hello $name"
```

This takes input from the user and prints a message.

---

### 7. How to create a user account?

In UNIX, user accounts are created using the useradd command (requires root privileges).
Example:

```
sudo useradd ayush

sudo passwd ayush
```

The first command creates a new user named “ayush”, and the second sets a password. The system also creates a home directory (if specified) and assigns default configuration files. You can verify the account in /etc/passwd.

---

### 8. How to check your current group?

You can check your current group and group memberships using the commands:

```
groups
```

or

```
id -gn
```

---

### 9. How to create a new file in UNIX using vi command?

To create a new file using vi, type:

vi filename

This opens the vi editor. Press **i** to enter **insert mode**, type the content, and then press **Esc** to exit insert mode. To save and quit, type **:wq** and press Enter. If the file doesn't exist, vi automatically creates it upon saving.

---

### 10. What is the format for executing a command in UNIX?

In UNIX, the general syntax for executing a command is:

command [options] [arguments]

- **command** – the name of the program to run.
- **options** – modify the behavior of the command (usually preceded by - or --).
- **arguments** – specify files, directories, or parameters.
  Example:

ls -l /home

Here, ls is the command, -l is the option, and /home is the argument.

---

## 5 MARKS QUESTIONS

---

### 1. Name few significant features of UNIX operating system.

UNIX is a **powerful, multiuser, multitasking operating system** with several key features:

1. **Multitasking** – It can run multiple processes simultaneously without affecting performance.
2. **Multiuser capability** – Many users can log in and work at the same time without interference.
3. **Portability** – Written mostly in C language, making it easy to run on different hardware platforms.
4. **Security** – Provides file permissions (read, write, execute) for users and groups.
5. **Hierarchical file system** – Organizes files in a tree-like directory structure.
6. **Networking and communication** – Built-in support for TCP/IP networking and remote access (SSH, FTP).
7. **Shell scripting** – Allows automation of tasks through command scripts.

---

## 2. Describe link in UNIX.

A **link** in UNIX is a **reference or pointer** to a file. It allows multiple filenames to refer to the same data. There are **two types of links**:

- **Hard Link**:
  - Creates another name for an existing file.
  - Both files share the same inode and data blocks.
  - Deleting one does not affect the other.
  - Cannot link across different file systems.
  - Example: ln file1 file2
- **Soft (Symbolic) Link**:
  - Acts like a shortcut to another file.
  - Contains the path of the target file.
  - If the original file is deleted, the symbolic link becomes invalid.
  - Example: ln -s file1 link1
    Links improve file accessibility and flexibility without duplicating data.

---

## 3. Describe filter in UNIX.

A **filter** in UNIX is a command that takes input from **standard input (stdin)**, processes it, and sends the result to **standard output (stdout)**. Filters are commonly used with **pipes (|)** to process data step by step.
Examples of filters include:

- grep – searches text patterns.
- sort – sorts lines of text.
- wc – counts words, lines, and characters.
- uniq – removes duplicate lines.
- tr – translates or deletes characters.
  Example:

```
cat file.txt | grep "error" | sort | uniq
```

Filters make UNIX powerful for text processing and automation.

---

## 4. What are the different types of UNIX files?

UNIX treats everything as a **file**, including devices and processes. The main types of UNIX files are:

1. **Regular files** – Contain text, data, or executable programs.
2. **Directory files** – Contain other files and subdirectories (like folders).
3. **Special device files** – Represent hardware devices such as printers, terminals, or disks.
    - **Character device files** – Data is transmitted character by character (e.g., keyboard).
    - **Block device files** – Data is transmitted in blocks (e.g., hard disk).
4. **FIFO (Named Pipe)** – Enables inter-process communication by passing data in sequence.
5. **Socket files** – Used for network communication between processes.
6. **Symbolic links** – Pointers to other files or directories.

---

## 5. What do chmod, chown and chgrp commands do in UNIX?

These commands manage **file permissions and ownership**:

- **chmod (Change Mode):**
    - Used to modify file permissions (read, write, execute).
    - Syntax: chmod [permissions] filename
    - Example: chmod 755 script.sh (gives full permission to owner, read/execute to others).
- **chown (Change Owner):**
    - Changes the ownership of a file or directory.
    - Syntax: chown [new_owner] filename
    - Example: chown root file.txt
- **chgrp (Change Group):**
    - Changes the group ownership of a file.
    - Syntax: chgrp [groupname] filename
    - Example: chgrp staff report.txt
      These commands are vital for maintaining **system security and access control**.

---

## 6. What is a zombie process in UNIX? How do you find it?

A **zombie process** is a process that has **completed execution** but still remains in the **process table** because its **parent has not read its exit status**.

- It has a status **"Z"** (for zombie) in process listings.
- It doesn't consume CPU or memory but occupies a process ID.
- Occurs when the parent process neglects to call wait() after a child terminates.

**To find zombie processes:**

```
ps aux | grep Z
```

or

```
top
```

You can identify them by the **Z** status and terminate the parent process if needed.

---

## 7. Explain system boot up in UNIX.

The **UNIX boot-up process** involves several stages:

1. **BIOS/POST** – Hardware is checked and initialized.
2. **Bootloader (e.g., GRUB)** – Loads the UNIX kernel from disk into memory.
3. **Kernel Initialization** – Kernel configures devices, mounts the root filesystem, and starts essential system processes.
4. **init or systemd process** – The first user-space process started by the kernel. It reads configuration files like /etc/inittab or /etc/systemd/.
5. **Runlevel Scripts** – Execute startup scripts for background services (networking, logging, daemons).
6. **Login Prompt** – Finally, the system displays a login prompt (CLI or GUI) for user interaction.

---

## 8. Name different types of shell available in UNIX.

UNIX provides multiple shells, each with distinct syntax and features:

1. **Bourne Shell (sh)** – Original UNIX shell; simple and fast.
2. **C Shell (csh)** – Uses C-like syntax and includes command history and aliasing.
3. **Korn Shell (ksh)** – Combines Bourne's simplicity with C shell's features.
4. **Bash (Bourne Again Shell)** – Most popular and default in Linux; supports scripting and auto-completion.
5. **Z Shell (zsh)** – Advanced shell with enhanced customization and plugin support.

6. **Fish Shell (Friendly Interactive Shell)** – User-friendly and modern syntax.
   Each shell acts as a **command interpreter and scripting environment**.

---

### 9. How many types of control instructions are available in shell scripting?

Shell scripting supports several **control instructions** to control the flow of execution:

1. **Sequence Control** – Commands executed in order, separated by ; or new lines.
2. **Decision Control (Conditional statements)** – if, if-else, case statements control execution based on conditions.
3. **Loop Control** – Repetition using loops like for, while, until.
4. **Switch Control** – Implemented through case statements to handle multiple choices.
5. **Break and Continue** – Used to alter loop execution flow.
   These control structures make shell scripts **dynamic and logic-driven**.

---

### 10. Write difference between grep and find command in table format.

| Basis | grep Command | find Command |
|---|---|---|
| **Purpose** | Searches for a pattern within file contents. | Searches for files and directories based on conditions. |
| **Search Target** | Works on **text within files**. | Works on **file names and attributes**. |
| **Syntax** | grep [pattern] [filename] | find [path] [options] [expression] |
| **Example** | grep "error" logfile.txt | find /home -name "*.txt" |
| **Output** | Displays matching lines containing the pattern. | Displays the **pathnames** of matching files. |
| **Use Case** | Content search inside files. | File search based on name, type, size, or date. |

---

# 8 MARKS QUESTIONS

---

## 1. Partition Creation in UNIX OS

## (or) Describe UNIX Booting Process Step by Step

---

### Partition Creation in UNIX OS

Partitioning in UNIX refers to the process of dividing a single physical hard disk into multiple logical sections known as **partitions**. Each partition behaves as an independent unit with its own file system. Partitioning allows better data organization, system management, and performance optimization. UNIX systems require at least one partition to store the operating system and one swap partition to handle virtual memory operations.

When UNIX is installed, the administrator divides the disk into sections such as **root (/)**, **/home**, **/var**, and **/usr**. This separation ensures that user data and system files remain isolated, enhancing system stability and reducing the risk of data loss during system failures.

The process of creating a partition involves several systematic steps. First, the system administrator lists the available disks using the fdisk -l command to identify the disk to be partitioned. Once identified, the fdisk utility is executed, for example, sudo fdisk /dev/sda. This command opens the partition table of the selected disk. Inside the tool, the administrator presses n to create a new partition, selects the partition type — **primary**, **extended**, or **logical** — and defines its size. After the new partition is created, the w command is used to write changes permanently to the disk.

Once created, the partition must be formatted using a file system. Common file systems used in UNIX are **ext4**, **xfs**, and **btrfs**. For example:

mkfs.ext4 /dev/sda1

This command formats the partition /dev/sda1 with the ext4 file system. After formatting, it must be **mounted** to a directory using:

mount /dev/sda1 /mnt/data

Finally, to ensure the partition mounts automatically on system startup, an entry is added to the /etc/fstab file.

Partitioning is crucial for UNIX systems because it helps maintain **system efficiency**, **data integrity**, and **security**. It allows independent management of user data, simplifies backup and recovery, and makes the system more organized and reliable.

---

### UNIX Booting Process (Step-by-Step)

The UNIX booting process is a sequence of operations that occur from the moment the system is powered on until the user is presented with a login prompt. The process involves several key stages, each responsible for initializing hardware and software components.

1. **BIOS/POST Stage:**
   When the computer is powered on, the Basic Input/Output System (BIOS) or Unified

Extensible Firmware Interface (UEFI) performs the Power-On Self-Test (POST) to verify hardware integrity, including RAM, CPU, and storage devices.

2. **Boot Loader Stage:**
   After successful POST, control passes to the **boot loader**, typically GRUB (Grand Unified Bootloader) in UNIX systems. The boot loader resides in the Master Boot Record (MBR) and is responsible for locating and loading the UNIX kernel into memory. It also allows users to select between multiple operating systems if present.

3. **Kernel Initialization:**
   Once loaded, the kernel becomes the central component of the operating system. It initializes low-level hardware, sets up memory management, mounts the root file system, and starts device drivers. The kernel also initializes process scheduling and basic input/output systems.

4. **init or systemd Process:**
   After the kernel is initialized, it starts the first user-space process — traditionally init (process ID 1), or in modern systems, systemd. This process acts as the parent of all user-level processes and controls system startup.

5. **Runlevel Configuration:**
   The init process reads configuration files such as /etc/inittab or /etc/systemd/system/. Depending on the runlevel or target (multi-user, graphical, etc.), it executes startup scripts located in directories like /etc/rc.d/ or /lib/systemd/.

6. **Daemon and Service Initialization:**
   Various background services (daemons) like network, cron, and syslog are started to provide networking, scheduling, and logging functionalities.

7. **Login Stage:**
   Finally, the getty or login process displays a login prompt on the terminal or graphical interface, enabling user interaction.

In summary, the UNIX booting process transforms an inactive machine into a fully operational environment by sequentially initializing hardware, kernel, and user services.

---

## 2. File Management in UNIX

## (or) User and Group Management in UNIX

---

### File Management in UNIX

File management is a fundamental aspect of the UNIX operating system. In UNIX, **everything is treated as a file**, including devices, processes, and directories. File management involves the creation, organization, access, and manipulation of files on the system.

UNIX follows a **hierarchical file system** structure that begins with the **root directory ("/")**, which acts as the top of the directory tree. All other directories and files branch from it. For example, /bin stores essential binaries, /home contains user directories, and /etc holds configuration files.

Each file in UNIX has attributes such as **name, inode number, permissions, ownership, size, and timestamps**. These attributes are essential for managing access control and maintaining file integrity. Users can view these attributes using the ls -l command.

There are several **types of files** in UNIX:

- **Regular files**: Contain user data, text, or program binaries.
- **Directory files**: Contain information about other files.
- **Character and block device files**: Represent hardware devices.
- **Pipes and sockets**: Facilitate inter-process communication.
- **Symbolic links**: Act as shortcuts to other files.

UNIX supports different **access mechanisms**:

1. **Sequential Access** – Data is accessed in sequence, useful for reading logs or text files.
2. **Direct (Random) Access** – Allows jumping to any part of the file; used in databases.
3. **Indexed Access** – Uses index tables for fast data retrieval, often used in large data systems.

Common file management commands include cp, mv, rm, cat, more, and vi. System administrators also use tools like fsck (file system check), du (disk usage), and df (disk free space) for maintenance.

File management in UNIX ensures organized storage, efficient retrieval, and strong security, making it a reliable operating system for multi-user environments.

---

## User and Group Management in UNIX

UNIX is a **multi-user operating system**, and managing users and groups is a crucial part of maintaining system security and control.

Each user is assigned a **unique user ID (UID)** and belongs to at least one **group**, identified by a **group ID (GID)**. This structure enables fine-grained control over file permissions and resource usage.

To create a new user, the system administrator uses:

sudo useradd username

sudo passwd username

This command creates an entry for the user in /etc/passwd and sets a password stored (encrypted) in /etc/shadow.

Groups are used to categorize users who share similar access needs. For instance, all developers can be added to a group named developers. Groups are managed with:

sudo groupadd developers

sudo usermod -aG developers username

The /etc/group file maintains information about group memberships. A user can belong to multiple groups to inherit combined permissions.

Effective user and group management in UNIX ensures **system stability, security, and efficient access control**. It allows administrators to enforce policies, monitor activity, and maintain accountability across multiple users.

---

## 3. Shell and Vi Editor

## (or) System Call and Pipe Example

---

### Shell and Vi Editor

A **shell** in UNIX is a command interpreter that acts as a bridge between the user and the kernel. It interprets user commands, executes them, and displays the output. Popular shells include Bourne Shell (sh), C Shell (csh), Korn Shell (ksh), and Bash (bash). The shell also allows scripting, where users can write sequences of commands in a file (a shell script) to automate repetitive tasks.

The **Vi editor** is one of UNIX's most widely used text editors. It is powerful, lightweight, and available on all UNIX systems. Vi operates in **three modes**:

1. **Command Mode** – The default mode upon opening a file. It allows users to navigate, copy, delete, and paste text using commands like dd (delete line), yy (copy line), and p (paste).
2. **Insert Mode** – Entered by pressing i, a, or o. In this mode, the user can type and edit text directly.
3. **Last Line (Ex) Mode** – Activated by pressing :. It allows commands like :w (save), :q (quit), and :wq (save and quit).

**Example:**

vi test.txt

# Press i → type text → Esc → :wq → Enter

Vi provides robust editing capabilities even in systems without a graphical interface. Its modal design allows precise and fast text manipulation, making it an essential tool for UNIX administrators.

---

### System Call and Pipe Example

System calls are the fundamental interface between a user program and the UNIX kernel. They enable programs to request services such as file manipulation, process control, or communication. Examples include read(), write(), fork(), exec(), and exit().

A **pipe** is a type of system call used for **inter-process communication (IPC)**. It allows data to flow from one process to another in a unidirectional manner. For instance, in a command like:

ls | sort

The output of ls is sent directly to sort through a pipe.

**Example in C:**

```
#include <unistd.h>
#include <stdio.h>

int main() {
   int fd[2];
   pipe(fd);
   if (fork() == 0) {
      close(fd[1]);
      char buffer[50];
      read(fd[0], buffer, sizeof(buffer));
      printf("Child read: %s\n", buffer);
   } else {
      close(fd[0]);
      write(fd[1], "Hello UNIX!", 11);
   }
   return 0;
}
```

---

## 4. Shell Script to Merge Files and Sort Output

## (or) Short Notes on Utility Programs

---

### Shell Script

A shell script automates the merging and sorting of multiple files. The following script merges three files, sorts their combined contents, and displays them page by page.

```
#!/bin/bash
# Script to merge and sort contents of three files and display page by page

echo "Enter three filenames:"
read f1 f2 f3
```

```
cat $f1 $f2 $f3 > temp.txt

sort temp.txt > result.txt

echo "Sorted and merged content:"

more result.txt
```

**Explanation:**

- cat combines file contents into a single file.
- sort arranges lines alphabetically or numerically.
- more allows viewing the file one screen at a time.
  This script demonstrates automation and data processing capabilities of shell scripting.

---

## Detailed Note on UNIX Utility Programs

In UNIX operating systems, **utility programs** (or utilities) are small but powerful tools designed to perform specific system or file-handling tasks efficiently. They are an essential part of the UNIX philosophy — *"Do one thing and do it well."* These programs help users perform operations such as combining files, filtering data, text processing, and system maintenance directly from the command line or within shell scripts.

Utility programs make UNIX extremely flexible, as they can be used alone or combined using **pipes (|)** and **redirection (>, <)** to perform complex operations with simple commands. Some of the most commonly used text and file manipulation utilities include **paste**, **join**, **uniq**, and **tr**.

---

### 1. paste Command

The paste utility merges lines from multiple files **horizontally**, separating fields by a tab character by default. It is mainly used when we want to combine corresponding lines of two or more files into a single output line.

**Syntax:**

```
paste [options] file1 file2 ...
```

**Example:**

Suppose we have two files:

**file1.txt**

```
A

B

C
```

**file2.txt**

1

2

3

Command: paste file1.txt file2.txt

**Output:**

A 1

B 2

C 3

**Options:**

- -d : Specifies a custom delimiter instead of a tab.
  Example:
- paste -d "," file1 file2

Output:

A,1

B,2

C,3

- -s : Serial mode — merges entire file contents line by line in sequence.

**Use Case:**

paste is useful in report generation, merging related data from two files, or creating CSV-like outputs from plain text files.

---

**2. join Command**

The join command combines lines from two files **based on a common field** (like a database join operation). It reads both files and merges the lines that share a matching field, often used for data comparison or database-like text processing.

**Syntax:**

join [options] file1 file2

**Conditions:**

- Both files must be **sorted** on the joining field.
- By default, join matches **the first field** of each file.

**Example:**

**file1.txt**

1 Ram

2 Sita

3 Rohan

**file2.txt**

1 Delhi

2 Mumbai

3 Kolkata

Command: join file1.txt file2.txt

**Output:**

1 Ram Delhi

2 Sita Mumbai

3 Rohan Kolkata

**Options:**

- -1 and -2: Specify which fields to join from each file.
  Example: join -1 2 -2 1 f1 f2
- -t : Define a custom delimiter.

**Use Case:**

Useful for combining employee IDs with their department names, student IDs with marks, or any related datasets stored in plain text format.

---

**3. uniq Command**

The uniq utility is used to **remove duplicate lines** from a sorted file or to report repeated lines. It compares **adjacent lines** and filters out repeated occurrences.

**Syntax:**

uniq [options] [input-file] [output-file]

**Example:**

**names.txt**

Aayush

Aayush

Rohan

Riya

Riya

Command: uniq names.txt

**Output:**

Aayush

Rohan

Riya

However, uniq works correctly **only if the file is sorted**, so it's often used with sort:

sort names.txt | uniq

**Options:**

- -c : Prefix each output line with the count of occurrences.
  Example:
- sort names.txt | uniq -c

Output:

2 Aayush

1 Rohan

2 Riya

- -d : Display only duplicate lines.
- -u : Display only unique lines.

**Use Case:**

Used for data cleaning, removing redundancy in lists, and summarizing occurrences of repeated items.

---

**4. tr Command (Translate Command)**

The tr command (translate) is a very powerful utility used to **translate, replace, squeeze, or delete characters** from input text. It works on a stream of characters and is often used in combination with pipes.

**Syntax:**

tr [options] set1 [set2]

**Example 1 – Change Case:**

Convert lowercase letters to uppercase:

tr 'a-z' 'A-Z' < file.txt

If file.txt contains:

hello unix

Output:

HELLO UNIX

**Example 2 – Delete Characters:**

Remove all digits from a file:

```
tr -d '0-9' < data.txt
```

**Example 3 – Squeeze Repeated Spaces:**

```
tr -s ' ' < file.txt
```

This replaces multiple spaces with a single space.

**Options:**

- -d : Delete characters in the specified set.
- -s : Squeeze repeated occurrences of characters.
- -c : Complement the set (apply to all characters *except* those specified).

**Use Case:**

tr is useful for text normalization, formatting data, preparing input for scripts, and cleaning unwanted symbols or whitespaces.

---

**Importance of UNIX Utilities**

These utilities demonstrate the **power and simplicity of the UNIX environment**. Instead of using large, complex programs, UNIX promotes combining multiple small utilities through **pipes** to achieve complex processing. For example:

```
cat file1 file2 | sort | uniq | tr 'a-z' 'A-Z'
```

This single command merges two files, sorts them, removes duplicates, and converts text to uppercase — showcasing the efficiency of UNIX command-line design.

---